

Search by meaning

How the prototype works, what it measures, and what breaks when the collection gets large.

The problem

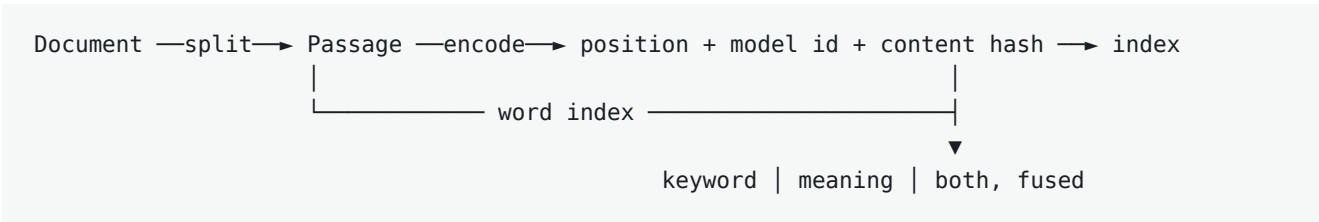
Keyword search — BM25, Postgres full-text, Elasticsearch out of the box — ranks documents by term overlap. If the searcher's words do not appear in the document, the document scores zero and is not returned at any depth. It is not ranked poorly; it is invisible.

That is the entire failure this addresses. Two measured examples from a 112-document test collection:

- "Norman conquest" returns nothing. Seven relevant documents exist; none contains either word.
- "why does bread rise" returns the article on **Inflation**, which contains "a sustained *rise* in the general price level". Term overlap without shared meaning.

How it works

Each document is split into passages, and each passage is converted into a list of numbers — a position in a space arranged so that text about similar things ends up nearby. A search query is converted the same way, and the closest passages are returned. No shared vocabulary is required.



The detail that matters most

Each passage has its document title attached before it is encoded. A passage reading "his army crossed the Channel in September" is nearly useless alone; with "The Battle of Hastings" attached it carries its subject. This one line does more for result quality than any other single decision in the pipeline, and costs nothing.

Three retrieval modes

MODE	RANKS BY	STRONG ON	BLIND TO
keyword	term overlap (BM25)	names, dates, figures, identifiers	anything phrased differently
meaning	closeness of position	paraphrase, concepts, questions	exact tokens it blurs together
fused	combined ranking	mixed traffic	—

Fusion combines the two lists by *rank*, not by score. Term-overlap scores are unbounded while closeness scores sit between -1 and 1 , so blending the numbers directly requires normalisation constants that must be re-tuned whenever the collection or the model changes. Ranks are comparable by construction.

What was measured

32 questions, hand-labelled with the documents that genuinely answer each, run against the same 112 documents.

MODE	RECALL@5	MRR@10	NDCG@10
keyword	0.355	0.625	0.415
meaning	0.841	0.984	0.890
fused	0.624	0.826	0.741

Read those as: *Recall@5* is the share of relevant documents appearing in the top five. *MRR@10* rewards getting one right answer to the top. *nDCG@10* discounts by rank across all relevant documents and is the one to trust when they disagree.

Per question, meaning-based retrieval gave the better result on **30 of 32**, keyword on 1, with 1 tie. Keyword scored exactly zero — nothing relevant at any depth — on **8**.

The result that went against the design. Fusing the two scored worse than meaning alone: 0.741 against 0.890. That is not a bug in the fusion. It is what rank fusion does when one input list is mostly empty, which is the case on a question set deliberately weighted towards phrasings keyword search cannot handle.

The honest reading is to ship meaning-based retrieval as the default for this kind of content, and revisit fusion against traffic containing more exact-name and identifier lookups, where term overlap earns its half of the merge. On a library catalogue — full of author, title and ISBN searches — we would expect fusion to win. That should be re-measured, not assumed.

On trusting these numbers. The test collection and the question set share an author, so the absolute values are indicative rather than authoritative. The gap between modes is the signal. A script is included that builds a collection from real Wikipedia articles for a non-circular re-run.

What breaks at scale

The encoding model is not the bottleneck people expect. Three other things are, and each has a specific countermeasure in the code.

1. Changing the model invalidates everything

Positions produced by two different models occupy different spaces. Comparing them returns confident nonsense with no error raised — the worst failure mode, because nothing looks wrong. Every stored position therefore carries its `model_id`, and both storage backends refuse a query encoded by a different model. That field is also what makes a rolling re-encode possible instead of taking search offline.

2. Re-indexing must not redo everything

Every passage carries a hash of its content. Re-running over an unchanged collection encodes nothing at all; a content update costs only the passages that actually changed. Without this, every edit costs the full collection price.

3. Storage is the real wall

1,000,000 PASSAGES	RAW SIZE	LEVER
1536 values, 4 bytes each	5.72 GB	—
truncated to 512 values	1.91 GB	3× smaller
stored at half precision	0.95 GB	a further 2×
384 values, half precision	0.72 GB	the self-hosted default

Six times smaller in total, which is the difference between needing a large managed database and fitting comfortably on a small one. Approximate-neighbour indexing keeps query time sub-linear as the collection grows.

Thin catalogue records

A library catalogue typically holds title, author and subject headings — around fifteen words per item. That is not enough text to place meaningfully. We measured the cost by stripping the test collection down to exactly that and re-running the same questions:

CONTENT INDEXED	RECALL@5	NDCG@10
Full article text (ceiling)	0.841	0.890
Thin records only	0.718	0.789

The aggregate drop understates it, because these titles are unusually descriptive. Per question the damage is severe and highly specific:

QUESTION	FULL TEXT	THIN
planning a landing on a defended coast	0.871	0.296
1066	0.876	0.369
how electricity reaches a house	1.000	0.581
Norman conquest	0.980	0.709

The pattern is consistent: **questions collapse exactly when the relevant documents' titles do not name the concept.** "1066" loses more than half because the Domesday Book, the Bayeux Tapestry, Stamford Bridge and Harold Godwinson never contain the string.

Closing that gap

A language model reads each record and writes a short description, which is indexed alongside the record rather than instead of it. This works because it can state concepts the record never contains — turning a subject heading of "Great Britain — History" into text naming the Norman Conquest, William the Conqueror and the Domesday survey. The vocabulary bridge is built deliberately instead of hoped for.

Three rules carry the risk:

- **Generated text is never shown to a reader.** Match against it, then display the real catalogue record. A library is an authority-bearing institution; this demotes a fabrication from a false statement by the library to a mildly poor search result.
- **The model declares whether it knows the work.** Given only a title and a subject heading it is drawing on training knowledge, not the record. If it recognises the work it may use that knowledge; if not, it may expand only what the record already says and must invent nothing.
- **The prompt is versioned like the model.** A prompt change invalidates every stored description, exactly as a model change invalidates every position — except regenerating costs money rather than processor time. Prompt version, model and a hash of the source record are stored with every description, so re-runs are incremental.

What is not proven

Three parts of the system have never run against their real dependencies. They are written and unit-tested against substitutes, which is not the same as working:

- Record enrichment against a live language-model API.
- The Postgres storage backend against an actual database.
- The two hosted encoding backends, which construct and validate correctly but have never made a request.

The single most valuable next step is small and cheap: run enrichment over the 112 thinned records at two or three quality levels and re-measure. That fills the one empty cell in the table above and settles the quality-versus-cost question with evidence rather than argument, for a few pounds.

Every figure in this document is a measurement taken from the working prototype, not an estimate. The prototype is 44 files and roughly 4,100 lines of Python with 59 tests, which run without a model, a network connection or an API key.